

IUTCDI - replication SDA

Lecture 02 - 4601.pdf

 PDF

read. dont say anything

Lecture 02 - 4601

clearly explain leader follower model

The **Leader-Follower model** (also called **Master-Slave replication**) is a way databases keep copies of the same data on multiple servers.

Think of it like this:

- One server is the **Leader**
- Other servers are **Followers**

The leader is the **main authority** for writes.

Basic Idea

When users change data:

1. The write goes to the **Leader**
2. The leader saves the change
3. The leader sends the change to all followers
4. Followers copy the same change

So all replicas eventually contain the same data.

Simple Example

Imagine a social media app.

You update your profile picture.

Step-by-step

1. Client sends update

SQL

```
UPDATE users
SET picture_url = "new.jpg"
WHERE user_id = 1234;
```

This goes to the **Leader**.

2. Leader stores it

Leader updates its own database.

Now the leader has:

```
picture_url = new.jpg
```

3. Leader replicates the change

Leader sends the update to followers.

Followers also update their databases.

Reading Data

Clients can read from:

- the leader
- or followers

Usually:

- **Writes** → **Leader**
- **Reads** → **Followers**

This helps scale the system.

Why Use This Model?

1. High Availability

If one follower crashes, system still works.

2. Faster Reads

Many followers can serve read requests.

Instead of one machine handling everything:

1 Leader + 5 Followers

all followers help answer queries.

3. Geographic Distribution

Followers can be placed near users.

Example:

- Leader in USA
- Followers in Europe and Asia

Asian users read from Asian follower → lower latency.

Important Concept: Replication

Replication means:

keeping copies of the same data on multiple machines.

The leader sends a stream of changes to followers.

This stream is called:

- replication log
 - change stream
 - WAL stream (depending on database)
-

Synchronous vs Asynchronous Replication

This is extremely important.

1. Synchronous Replication

Leader waits for follower confirmation before saying:

"Write successful"

Flow:

Client → Leader → Follower confirms → Client gets success

Advantage

Very safe.

If leader crashes immediately,
follower already has the latest data.

Disadvantage

Slower.

If follower is slow/down,
writes block.

2. Asynchronous Replication

Leader does NOT wait.

Flow:

```
Client → Leader → Client gets success
           ↓
        follower updated later
```

Advantage

Fast.

System keeps working even if followers lag.

Disadvantage

Possible data loss.

If leader crashes before followers receive latest updates,
those writes disappear.

Replication Lag

Followers may be behind the leader for some time.

This delay is called:

Replication Lag

Example:

Leader has:

balance = 500

Follower still has:

balance = 300

because update hasn't arrived yet.

Problems Caused by Replication Lag

1. Read-After-Write Problem

You update your post,
then refresh page immediately.

Follower still has old data.

You think:

"My post disappeared!"

2. Monotonic Read Problem

You first read new data from one follower,
then older data from another follower.

Time appears to go backward.

3. Consistent Prefix Problem

You may see:

Answer
Question

instead of:

Question
Answer

because replicas are updated at different speeds.

What Happens If Leader Fails?

This is called:

Failover

Steps:

1. Detect leader failure
2. Promote one follower to new leader
3. Redirect clients to new leader

4. Other followers follow new leader

Problems During Failover

Data Loss

If replication was asynchronous,
new leader may miss latest writes.

Split Brain

Two nodes both think:

"I am the leader"

Very dangerous.

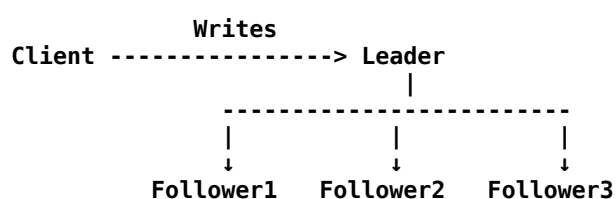
Can corrupt data.

Real Databases Using Leader-Follower

Examples:

- PostgreSQL
- MySQL
- MongoDB
- SQL Server
- Oracle
- Kafka
- RabbitMQ

Visual Model



Reads can go to followers

Core Characteristics

Feature	Leader-Follower
Writes	Only leader
Reads	Any replica
Scaling reads	Easy
Scaling writes	Hard
Consistency	Depends on replication
Failure handling	Requires failover

Main Tradeoff

Leader-Follower replication trades:

strong consistency
vs
availability + performance

The faster and more available you make it,
the more stale reads you may get.

One-Line Summary

In the Leader-Follower model, one node accepts writes and propagates changes to replica nodes that mainly serve reads.

explain Synchronous Versus Asynchronous Replication from the pdf

In the PDF, **Synchronous vs Asynchronous Replication** explains **how the leader communicates updates to followers** in a replicated database system.

Lecture 02 - 4601

The main question is:

After the leader receives a write, should it wait for followers before confirming success to the client?

That creates two replication styles:

1. Synchronous Replication

In **synchronous replication**, the leader waits until the follower confirms it received the update before telling the client:

"Write successful."

Flow

```
Client → Leader → Follower
          ↓
        follower confirms
          ↓
    Leader replies SUCCESS
```

Example

Suppose you update your profile picture.

```
SQL
UPDATE users
SET picture = "new.jpg"
WHERE id = 1234;
```

What happens?

Step 1

Client sends request to leader.

Step 2

Leader writes data locally.

Step 3

Leader sends update to follower.

Step 4

Leader WAITS.

It does NOT yet tell the client the write succeeded.

Step 5

Follower replies:

OK

Step 6

Now leader tells client:

Write successful

Key Property

The follower is guaranteed to already have the newest data.

So if the leader suddenly crashes:

✓ follower still has latest update

This improves **durability** and **consistency**.

If the synchronous follower becomes:

- then the leader cannot continue processing writes.

Everything blocks.

Lecture 02 - 4601

Imagine 10 followers.

If leader waits for ALL followers:

1 slow follower = entire system slows

So databases usually make:

- This is called:

Lecture 02 - 4601

In asynchronous replication, the leader does NOT wait for followers.

The leader immediately replies to the client after writing locally.

```
Client → Leader → SUCCESS immediately
                  ↓
                followers updated later
```

Same profile picture update.

Step 1

Client sends request.

Step 2

Leader stores update locally.

Step 3

Leader instantly replies:

SUCCESS

Step 4

Later, followers receive update.

Could be:

- milliseconds later
- seconds later
- minutes later

depending on system conditions.

Lecture 02 - 4601

Why Lag Happens

Followers may fall behind because of:

- network problems
- high traffic
- server overload
- follower recovery after crash

The PDF says lag can sometimes become several minutes.

Lecture 02 - 4601

Major Advantage

Very fast and highly available.

Even if all followers are down:

✓ leader can still process writes

This is why asynchronous replication is widely used.

Lecture 02 - 4601

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Major Disadvantage

Possible data loss.

Suppose:

1. Leader accepts a write
2. Tells client "success"
3. Before followers receive update, leader crashes permanently

Then:

✗ latest write disappears forever

because followers never received it.

Lecture 02 - 4601

Core Difference

Synchronous	Asynchronous
Leader waits for follower	Leader does not wait
Safer	Faster
Higher consistency	Higher availability
Slower writes	Faster writes
Less chance of data loss	Possible data loss
Can block if follower fails	Continues even if followers fail

Real-World Analogy

Synchronous

You send an important document and WAIT for confirmation:

"Did you receive it?"

Only after hearing "yes" do you relax.

Asynchronous

You send the document and move on immediately.

You assume delivery will happen later.

Faster, but riskier.

Important Insight from the PDF

The PDF emphasizes:

weakening durability may sound bad, but asynchronous replication is still widely used because it gives better performance and availability.

Lecture 02 - 4601

That is one of the biggest themes in distributed systems:

Tradeoff

Consistency & Durability
vs
Performance & Availability

You usually cannot maximize both simultaneously.

Setting Up New Followers from slide

Setting Up New Followers

This section explains:

How to add a new follower replica to a running replicated database system without stopping the database.

Lecture 02 - 4601

Why Do We Need New Followers?

A system may add new followers for several reasons:

- increase read scalability
- replace failed machines
- improve availability
- create backups
- distribute data geographically

But there is a challenge.

The Problem

The database is constantly changing.

Clients are continuously:

- inserting data

- updating records
- deleting records

So if you simply copy database files directly:

copy database folder → new server

the copied data may become inconsistent.

Why?

Because while copying:

- one part of database may represent time T1
- another part may represent time T2

The database state becomes mixed and invalid.

Lecture 02 - 4601

Bad Solution: Stop the Database

One solution could be:

LOCK DATABASE

Stop all writes,
copy files,
then restart.

But this causes downtime.

And high availability systems avoid downtime.

Correct Solution

The PDF describes a safer process.

Step-by-Step Process

Step 1 — Take a Consistent Snapshot

First, create a snapshot of the leader database at one exact moment in time.

Lecture 02 - 4601

A snapshot means:

frozen copy of database state

Important:

- snapshot should be internally consistent
- preferably without locking the whole database

Most modern databases support this.

Examples:

- PostgreSQL backups
- MySQL tools like `innobackupex`

Why Snapshot Matters

Suppose database contains:

Account A = 100
Account B = 200

During a transfer:

A → 50
B → 250

If snapshot captures only half the transaction:

A = 50
B = 200

data becomes corrupted.

So snapshot must represent one exact consistent point in time.

Step 2 — Copy Snapshot to New Follower

Now move the snapshot to the new follower machine.

Lecture 02 - 4601

At this moment:

Follower = old state of leader

because leader continued processing new writes after snapshot.

So follower is still behind.

Step 3 — Replay Missing Changes

Now the follower connects to the leader and asks:

“Give me everything that happened after the snapshot.”

To do this, the snapshot stores a special replication position.

Examples from the PDF:

- PostgreSQL → Log Sequence Number (LSN)
- MySQL → Binlog Coordinates

Lecture 02 - 4601

What Is Replication Log Position?

The leader keeps a replication log:

```
Write 1
Write 2
Write 3
Write 4
...
```

Suppose snapshot was taken after:

```
Write 100
```

Then follower requests:

```
Send writes 101 onward
```

Leader streams all missing updates.

Step 4 — Catch Up

Follower applies all missing writes one by one.

Lecture 02 - 4601

Eventually:

```
Follower state = Leader state
```

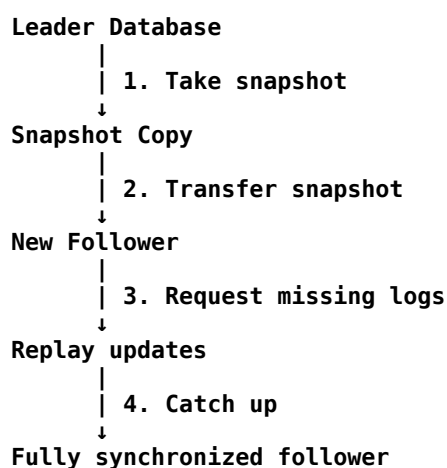
Now the follower is fully synchronized.

This is called:

Catching Up

After that, follower continues receiving live updates normally.

Visual Flow



Important Concept

The follower does NOT start from scratch by replaying entire database history.

Instead:

```
Snapshot + recent logs
```

This is much faster.

Main Goal

The whole process tries to achieve:

- ✓ No downtime
- ✓ Consistent data
- ✓ Accurate replica
- ✓ Continuous availability

Key Insight from the PDF

The PDF emphasizes that:

setting up followers can usually be done without downtime.

Lecture 02 - 4601

That is a major advantage of modern replication systems.

One-Line Summary

A new follower is created by taking a consistent snapshot of the leader, copying it to the new server, and replaying all updates that happened after the snapshot until the follower catches up with the leader.

handling node outages

Handling Node Outages

This section explains:

How a replicated database system continues working even when some machines fail.

Lecture 02 - 4601

In distributed systems, node failures are normal.

A server may fail because of:

- hardware crash
- power outage
- network issue
- reboot for maintenance
- software bug

The goal is:

Keep the system running despite failures.

Two Types of Failures

The PDF discusses two cases:

1. **Follower failure**
2. **Leader failure**

Leader failure is much harder.

1. Follower Failure — Catch-Up Recovery

This is the easier case.

Lecture 02 - 4601

What Happens?

Suppose we have:

Leader
Follower A
Follower B

Now Follower B crashes.

While it is offline:

- leader continues processing writes
 - other followers continue working
-

How Recovery Works

Each follower stores a replication log position locally.

Example:

Last processed log = 500

After restarting, follower says to leader:

Send me everything after log 500

Leader sends missing updates.

Follower applies them one by one.

Eventually it catches up.

This process is called:

Catch-Up Recovery

Lecture 02 - 4601

Why This Is Easy

Because follower already knows:

where it stopped

So it only needs missing updates.

No complicated coordination needed.

Visual Example

Leader log:

1 2 3 4 5 6 7 8 9

Follower crashed after 5

After restart:

request → 6 7 8 9

Now follower becomes synchronized again.

2. Leader Failure — Failover

This is much more difficult.

Lecture 02 - 4601

If leader crashes:

No node can accept writes

because only leader handles writes.

So system must choose a new leader.

This process is called:

Failover

Main Steps of Failover

The PDF describes 3 major steps.

Step 1 — Detect Leader Failure

System must determine:

Is the leader really dead?

This is surprisingly hard.

How Detection Usually Works

Nodes continuously exchange heartbeat messages.

Example:

"Are you alive?"

If no response arrives within timeout:

Leader assumed dead

Example timeout:

30 seconds

Lecture 02 - 4601

Problem

Maybe leader is not dead.

Maybe:

- network is slow
- packets delayed
- temporary overload occurred

Incorrect detection can cause unnecessary failover.

Step 2 — Choose New Leader

Now one follower must become the new leader.

Lecture 02 - 4601

Usually the best candidate is:

Most up-to-date follower

because it minimizes data loss.

Leader selection may happen using:

- election algorithms
 - controller node
 - consensus protocols
-

Example

Suppose:

Follower A → latest log 1000
Follower B → latest log 980

Follower A is better candidate.

Step 3 — Reconfigure System

After choosing new leader:

1. Clients must send writes to new leader
2. Other followers must replicate from new leader
3. Old leader must stop acting as leader if it returns later

Lecture 02 - 4601

Important Danger: Split Brain

Suppose old leader comes back online.

But it still thinks:

"I am the leader"

Now two leaders exist simultaneously.

This is called:

Split Brain

Very dangerous.

Lecture 02 - 4601

Why Split Brain Is Dangerous

Both leaders may accept writes:

Leader 1 → value = A
Leader 2 → value = B

Data becomes inconsistent.

Possible outcomes:

- corruption
 - lost writes
 - conflicting records
-

Problem with Asynchronous Replication

The PDF highlights another major issue.

Lecture 02 - 4601

Suppose:

1. Leader accepts write
2. Write not yet replicated
3. Leader crashes
4. Follower becomes new leader

Then latest write disappears forever.

Example

Leader received:
balance = 500

but follower still has:

balance = 300

Leader crashes before replication.

New leader uses old value.

Result:

✗ recent write lost

GitHub Example from PDF

The PDF mentions a real GitHub incident.

Lecture 02 - 4601

An outdated MySQL follower became leader.

Problem:

- auto-increment IDs became inconsistent
- duplicate IDs were generated
- wrong private data shown to users

This shows how dangerous failover mistakes can be.

Timeout Tradeoff

Another important idea:

How long should system wait before declaring leader dead?

Long Timeout

- + fewer false failovers
- slower recovery

Short Timeout

- + faster recovery
- more false failovers

Temporary delays may wrongly trigger failover.

Why Handling Outages Is Hard

Distributed systems must deal with:

- unreliable networks
- delayed packets
- crashes
- partial failures
- inconsistent replicas

There is no perfect solution.

That's why some companies still prefer:

Manual failover

instead of fully automatic failover.

Lecture 02 - 4601

Visual Overview

```
Leader crashes
  ↓
Detect failure
  ↓
Choose new leader
  ↓
Redirect clients
  ↓
Followers sync with new leader
```

Core Idea

Handling outages is about:

maintaining availability
without losing too much consistency

This is one of the biggest challenges in distributed systems.

One-Line Summary

Follower failures are handled by replaying missed updates, while leader failures require failover: detecting failure, electing a new leader, and reconfiguring the system carefully to

avoid data loss and split brain.

explain Leader failure: Failoverpart clearly

Leader Failure: Failover

When the **leader node crashes**, the system loses the only node that accepts writes.

So the database must quickly choose another node to become the new leader.

This process is called:

Failover

Lecture 02 - 4601

Why Failover Is Needed

In leader–follower replication:

Writes → **Leader only**
Reads → **Followers**

If leader dies:

✗ no writes can happen

System becomes partially unavailable.

So another follower must take over.

Example Setup

Suppose we have:

Leader
Follower A
Follower B

Now leader crashes.

System must promote one follower:

Follower A → **New Leader**

Then all writes go to Follower A.

Main Steps of Failover

The PDF explains 3 major steps.

Lecture 02 - 4601

Step 1 — Detect That Leader Failed

System first needs to determine:

Is the leader really dead?

This sounds simple,
but it is actually difficult in distributed systems.

How Detection Works

Servers continuously send heartbeat messages:

"Are you alive?"

If leader stops responding for some timeout period:

Leader assumed failed

Example:

No response for 30 seconds

→ start failover

Lecture 02 - 4601

Important Problem

Maybe the leader is NOT actually dead.

Possible reasons:

- network delay
- temporary overload
- slow CPU
- packet loss

So the system may incorrectly think:

Leader failed

This creates dangerous situations.

Step 2 — Choose a New Leader

Now one follower must become the new leader.

This process may happen through:

- election algorithms
- voting
- controller nodes
- consensus systems

Which Follower Should Become Leader?

Usually:

Most up-to-date follower

Why?

Because it minimizes lost data.

Example

Suppose:

Follower A → log 1000
Follower B → log 950

Follower A is better candidate because it has newer data.

Step 3 — Reconfigure the System

After leader selection:

Clients

Must send writes to the new leader.

Followers

Must replicate from the new leader.

Old Leader

If old leader comes back online:

it must stop acting as leader

Otherwise two leaders may exist.

Biggest Problems During Failover

The PDF explains several dangerous issues.

Problem 1 — Data Loss

This happens mainly with:

Asynchronous Replication

Scenario

Suppose:

1. Leader receives a write
2. Leader confirms success to client
3. Leader crashes immediately
4. Followers never received latest update

Now new leader does not contain that write.

Result:

 data lost

Lecture 02 - 4601

Example

Leader has:

balance = 500

Follower still has:

balance = 300

Leader crashes before replication.

New leader becomes follower with old value.

Latest update disappears.

Problem 2 — Split Brain

One of the most dangerous distributed-system problems.

What Is Split Brain?

Two nodes both believe:

"I am the leader"

at the same time.

Lecture 02 - 4601

Why It Happens

Example:

- network partition occurs
- followers cannot contact leader
- followers elect new leader
- original leader is actually still alive

Now:

Old Leader + New Leader

both accept writes.

Why Split Brain Is Dangerous

Suppose:

Leader 1 → value = A
Leader 2 → value = B

Now replicas diverge.

Possible outcomes:

- corrupted data
 - conflicting writes
 - lost updates
-

Problem 3 — Wrong Timeout

Choosing timeout value is difficult.

Lecture 02 - 4601

Long Timeout

Example:

60 seconds

Advantage

Fewer false failovers.

Disadvantage

Slow recovery.

Users wait longer after real failure.

Short Timeout

Example:

5 seconds

Advantage

Fast recovery.

Disadvantage

Temporary network delays may trigger unnecessary failovers.

Why Unnecessary Failover Is Bad

Suppose system already has heavy load.

Then failover adds even more stress:

- leader election
- reconnection
- replication reconfiguration

This may worsen the outage.

Real GitHub Incident Mentioned in PDF

The PDF gives a real example.

Lecture 02 - 4601

GitHub promoted an outdated MySQL follower to leader.

Problem:

- auto-increment IDs became inconsistent
- duplicate IDs were generated
- Redis and MySQL became inconsistent
- private user data leaked incorrectly

This shows failover mistakes can cause serious real-world failures.

Visual Flow

```

Leader crashes
↓
Detect failure
↓
Choose best follower
↓
Promote new leader
↓
Redirect clients
↓
Followers sync with new leader
  
```

Core Tradeoff

Failover tries to balance:

Fast recovery
vs
Correctness & consistency

Recover too slowly → downtime

Recover too aggressively → corruption

Key Insight

The PDF emphasizes:

there are no easy solutions.

Lecture 02 - 4601

Leader failover is fundamentally difficult because distributed systems cannot perfectly distinguish between:

dead node
vs
temporarily unreachable node

One-Line Summary

Leader failover is the process of detecting leader failure, promoting a follower as the new leader, and reconfiguring the system—while carefully avoiding data loss, split brain, and unnecessary failovers.

explain Implementation of Replication Logs from the slave fully and in details and clearly

Implementation of Replication Logs

This section explains:

How the leader actually sends database changes to followers internally.

Lecture 02 - 4601

In leader-follower replication:

Leader receives writes



Followers must copy same changes

The key question is:

How are those changes transmitted?

The answer is:

Replication Logs

A replication log is a record of database changes that followers replay to stay synchronized.

Main Replication Methods

The PDF discusses 4 methods:

1. Statement-Based Replication
2. Write-Ahead Log (WAL) Shipping
3. Logical (Row-Based) Replication
4. Trigger-Based Replication

We'll go through each carefully.

1. Statement-Based Replication

This is the simplest idea.

Lecture 02 - 4601

Basic Idea

Leader sends the exact SQL statements executed by clients.

Example:

SQL

```
UPDATE users
SET balance = balance + 100
WHERE id = 5;
```

Leader sends this SQL command to followers.

Followers execute the same SQL.

Flow

```

Client → Leader executes SQL
      ↓
    SQL statement copied
      ↓
Followers execute same SQL
  
```

Why It Seems Good

Very simple.

Only SQL text needs to be transferred.

Efficient because SQL statements are usually small.

Big Problems

The PDF explains several situations where this breaks.

Lecture 02 - 4601

Problem 1 — Non-Deterministic Functions

Example:

```

SQL
UPDATE orders
SET created_at = NOW();
  
```

or:

```

SQL
SELECT RAND();
  
```

Each follower may generate different values.

Example

Leader time:

10:00:01

Follower time:

10:00:03

Now replicas become inconsistent.

Problem 2 — Auto-Increment Issues

Example:

```

SQL
INSERT INTO users(name)
VALUES('Imran');
  
```

Different followers may assign different IDs if execution order differs.

Problem 3 — Order Dependency

Example:

```
SQL
UPDATE accounts
SET balance = balance - 100
WHERE balance > 500;
```

If replicas already differ slightly,
same statement may produce different results.

Problem 4 — Side Effects

Triggers and stored procedures may behave differently on different replicas.

Example:

```
Trigger sends email
```

All followers might send duplicate emails.

Result

Statement-based replication becomes unreliable.

That's why modern systems usually avoid it.

Real Usage

The PDF says:

- Old MySQL versions used it
- VoltDB still uses it carefully

Lecture 02 - 4601

2. Write-Ahead Log (WAL) Shipping

This is much more important in real databases.

Lecture 02 - 4601

Core Idea

Instead of sending SQL statements:

```
send low-level storage changes
```

Leader copies its internal storage log directly to followers.

First: What Is a WAL?

Databases internally keep a special append-only log.

Whenever data changes:

change written to WAL first

before actual database pages are updated.

This helps recovery after crashes.

WAL Example

Suppose database changes:

balance: 300 → 500

WAL might record:

Byte 120 changed

Page 52 modified

Very low-level storage information.

Replication Using WAL

Leader:

1. Writes WAL locally
2. Sends WAL entries to followers

Followers replay identical WAL entries.

As a result:

followers become exact binary copies

of leader's storage structures.

Flow

Client write

↓

Leader WAL updated

↓

WAL entries streamed

↓

Followers replay WAL

Advantages

Very Fast

Followers directly apply storage-level changes.

No SQL parsing needed.

Exact Replica

Followers become almost identical copies.

Major Problem

The PDF emphasizes an important limitation.

Lecture 02 - 4601

WAL format is tightly coupled to storage engine internals.

What This Means

If database version changes internal storage format:

old WAL incompatible with new version

So:

✗ leader and follower often cannot run different DB versions.

Operational Problem

This makes upgrades difficult.

Without compatibility:

upgrade requires downtime

because all nodes must upgrade together.

Databases Using WAL Shipping

Examples:

- PostgreSQL
- Oracle

Lecture 02 - 4601

3. Logical (Row-Based) Replication

This solves WAL's tight coupling problem.

Lecture 02 - 4601

Core Idea

Instead of:

- SQL statements
- storage-level bytes

leader sends:

logical row changes

Example

Suppose:

SQL
INSERT INTO users(id,name)
VALUES(5, 'Imran');

Replication log contains something like:

Table: users
Action: INSERT
Row:
id=5
name=Imran

Types of Row Events

The PDF explains:

Insert

Log contains new row values.

Example:

INSERT:
id=5
name=Imran

Delete

Log contains enough information to identify deleted row.

Usually primary key.

Example:

DELETE:
id=5

Update

Log contains:

- row identifier
- changed values

Example:

```
UPDATE:  
id=5  
balance=500
```

Lecture 02 - 4601

Advantages

Decoupled from Storage Engine

Replication no longer depends on physical disk format.

So:

✓ leader and followers can run different DB versions

Easier Upgrades

Can upgrade followers first,
then promote them.

Minimal downtime.

Easier External Processing

Logical logs are human-readable.

Useful for:

- analytics
- caches
- search indexes
- data warehouses

This is called:

Change Data Capture (CDC)

Real Example

MySQL binlog in row-based mode uses this approach.

Comparison with WAL

WAL	Logical Replication
Physical storage changes	Logical row changes
Faster	More flexible
Storage-engine dependent	Storage-engine independent
Harder upgrades	Easier upgrades
Low-level bytes	Human-readable row events

4. Trigger-Based Replication

This pushes replication into application logic.

Lecture 02 - 4601

Core Idea

Database triggers detect changes.

Custom code handles replication.

What Is a Trigger?

A trigger is code automatically executed when data changes.

Example:

```
SQL
AFTER INSERT
AFTER UPDATE
AFTER DELETE
```

Example Workflow

Suppose user inserted:

```
SQL
INSERT INTO users ...
```

Trigger runs automatically:

log change into custom table

Then external process reads this table and replicates data elsewhere.

Advantages

Very flexible.

Can:

- replicate only specific tables
- transform data
- replicate to another database type
- implement custom conflict resolution

Disadvantages

The PDF says it has:

- higher overhead
- more bugs
- more complexity

Lecture 02 - 4601

Because replication logic is partly custom application code.

Real Tools Mentioned

- Oracle GoldenGate
- Databus
- Bucardo

Overall Comparison

Method	Sends	Advantages	Problems
Statement-Based	SQL queries	Simple	Non-determinism
WAL Shipping	Physical storage changes	Fast	Version coupling
Logical Replication	Row-level changes	Flexible	Slightly slower
Trigger-Based	Custom trigger events	Very flexible	Complex

Important Big Picture

The entire purpose of replication logs is:

keep followers synchronized with leader

But every method balances:

- performance
- flexibility
- consistency
- operational simplicity

Most Important Insight

The PDF gradually moves from:

high-level replication

↓

low-level physical replication

↓

logical replication

↓

application-controlled replication

Each step trades simplicity for flexibility.

One-Line Summary

Replication logs are the mechanism leaders use to propagate database changes to followers, and systems implement them using statement-based, physical WAL-based, logical row-based, or trigger-based replication depending on performance and flexibility requirements.

explain Problems with Replication Lag from the pdf

Problems with Replication Lag

This section explains one of the most important ideas in distributed systems:

Followers may not immediately contain the latest data written to the leader.

Lecture 02 - 4601

This delay is called:

Replication Lag

What Is Replication Lag?

In leader-follower replication:

Write → Leader first



Followers updated later

Followers need time to receive and apply updates.

That delay is:

Replication Lag

Why Lag Happens

The PDF says replication is usually fast:

less than 1 second

But under problems:

- network congestion
- heavy load
- follower recovery
- system overload

lag may become:

seconds or even minutes

Lecture 02 - 4601

Example

Suppose leader has:

balance = 500

Follower still has:

balance = 300

because update hasn't arrived yet.

Important Consequence

Two replicas may temporarily return different answers.

Example:

Leader → newest data

Follower → stale data

This temporary inconsistency is called:

Eventual Consistency

Lecture 02 - 4601

What Is Eventual Consistency?

It means:

If no new writes happen, eventually all replicas will become consistent.

Eventually = no guaranteed exact time.

Could be:

- milliseconds
- seconds
- minutes

Why Replication Lag Matters

The PDF explains that lag creates strange user-visible anomalies.

It discusses 3 major problems:

1. Reading Your Own Writes
2. Monotonic Reads
3. Consistent Prefix Reads

These are extremely important.

1. Reading Your Own Writes

Also called:

Read-After-Write Consistency

Lecture 02 - 4601

Problem Scenario

Suppose you post a comment.

Write goes to leader:

"My comment"

Immediately afterward you refresh page.

But your read request goes to a follower.

Follower still hasn't received update.

You see:

`comment missing`

You may think:

`"My post was lost!"`

Why This Happens

Because:

`write → leader`
`read → stale follower`

Follower lags behind.

Desired Guarantee

Users should always see their own updates immediately.

This is:

Read-After-Write Consistency

Meaning:

`after writing,`
`you must see your own write`

Solutions Mentioned in PDF

Solution 1 — Read Own Data from Leader

Example:

`your profile → always read from leader`
`other profiles → read from followers`

Because users mostly edit their own profile.

Solution 2 — Temporary Leader Reads

After a write:

`for 1 minute,`
`read from leader`

until followers likely catch up.

Solution 3 — Use Replication Lag Information

If follower is too stale:

don't use it for reads

Solution 4 — Timestamp Tracking

Client remembers last write timestamp.

System ensures reads come from replicas updated past that timestamp.

Cross-Device Problem

Suppose:

- you post from phone
- refresh from laptop

Now system must synchronize consistency across devices too.

The PDF says this becomes harder.

Lecture 02 - 4601

2. Monotonic Reads

This prevents:

Time Going Backward

Lecture 02 - 4601

Problem Scenario

Suppose user refreshes same page twice.

First Read

Request goes to fresh follower.

User sees:

New comment exists

Second Read

Request goes to more stale follower.

User now sees:

Comment disappeared

It appears as if:

time moved backward

Why This Happens

Different followers have different lag amounts.

Desired Guarantee

If user already saw newer data:

they should never later see older data

This is:

Monotonic Reads

Solution Mentioned

Route each user consistently to same replica.

Example:

hash(user_id) → choose replica

So user always reads from same follower.

Lecture 02 - 4601

3. Consistent Prefix Reads

This deals with:

Causality

Lecture 02 - 4601

Example from PDF

Conversation:

Mr. Poons:

"How far into the future can you see?"

Mrs. Cake:

"About ten seconds."

Clearly:

question happened before answer

Answer depends on question.

Problem with Replication Lag

Suppose:

- question replicated slowly
- answer replicated quickly

Another user may see:

Answer
Question

This makes no logical sense.

Why This Happens

Different partitions or replicas may update at different speeds.

Desired Guarantee

If writes happened in order:

A → then B

everyone should observe:

A before B

This is:

Consistent Prefix Reads

Particularly Important in Partitioned Databases

The PDF says this problem becomes worse when data is sharded across partitions.

Lecture 02 - 4601

Because different partitions replicate independently.

Big Picture

These problems occur because:

Replication is asynchronous

Followers are temporarily inconsistent.

Important Insight

Replication lag is not just technical.

It directly affects:

- user experience
 - correctness
 - application behavior
-

Solutions for Replication Lag

The PDF emphasizes:

applications must be designed carefully if eventual consistency can become noticeable.

Lecture 02 - 4601

General Strategies

Stronger Consistency Guarantees

Examples:

- read-after-write
 - monotonic reads
 - consistent prefix reads
-

Read from Leader

For critical reads.

Better Transactions

The PDF says transactions help hide many replication issues.

Lecture 02 - 4601

Core Tradeoff

Replication lag exists because systems prioritize:

availability + performance

over:

instant consistency

Visual Summary

```

Client writes
  ↓
Leader updated immediately
  ↓
Followers updated later
  ↓
Temporary inconsistency
  ↓
Eventually synchronized
  
```

Key Concept Map

Problem	What Goes Wrong
Read-after-write	User cannot see own update
Monotonic reads	Time appears backward
Consistent prefix reads	Events appear in wrong order

One-Line Summary

Replication lag occurs because followers receive updates asynchronously, causing temporary inconsistencies such as stale reads, disappearing data, and causally incorrect ordering until replicas eventually catch up with the leader.

explain Multi Leader replication properly in details

Multi-Leader Replication

So far, in leader-follower replication:

One leader handles all writes
Followers copy data

But this creates a limitation:

Only one node can accept writes.

If that leader becomes unreachable:

no writes possible

To solve this, systems sometimes use:

Multi-Leader Replication

Also called:

- master-master replication
- active-active replication

Lecture 02 - 4601

Core Idea

Instead of one leader:

multiple leaders can accept writes

Each leader:

- accepts client writes
- replicates changes to other leaders

Every leader also behaves like a follower of the others.

Basic Architecture

Example:

Datacenter A → Leader A
Datacenter B → Leader B
Datacenter C → Leader C

Each leader processes local writes.

Then leaders exchange updates asynchronously.

Visual Flow

```

Client → Leader A
      ↓
    replicate to B and C

Client → Leader B
      ↓
    replicate to A and C
  
```

All leaders continuously synchronize with each other.

Why Multi-Leader Exists

The PDF says multi-leader is useful mainly in special situations.

Lecture 02 - 4601

Usually NOT inside a single datacenter.

Because complexity becomes high.

Main Use Cases

The PDF explains 3 major use cases:

1. Multi-Datacenter Operation
 2. Offline Clients
 3. Collaborative Editing
-

1. Multi-Datacenter Operation

This is the most important use case.

Problem with Single-Leader Across Datacenters

Suppose:

Leader in USA
Users in Asia

Every write must travel across the internet to USA.

Result:

✗ high latency

Multi-Leader Solution

Place one leader in each datacenter.

Example:

USA → Leader A
Asia → Leader B
Europe → Leader C

Now Asian users write locally to Asian leader.

Much faster.

Internal Replication

Inside each datacenter:

normal leader-follower replication

Between datacenters:

leaders replicate to each other

Advantages of Multi-Datacenter Multi-Leader

The PDF explains 3 major benefits.

Advantage 1 — Better Performance

Writes happen locally.

So users avoid long intercontinental delays.

Single-Leader

Asia user → USA leader

High latency.

Multi-Leader

Asia user → Asia leader

Fast local writes.

Advantage 2 — Better Datacenter Failure Tolerance

Suppose USA datacenter crashes.

With single-leader:

must fail over

before writes continue.

With multi-leader:

other datacenters continue independently

No global outage.

Advantage 3 — Better Network Partition Tolerance

Inter-datacenter internet links may fail temporarily.

Single-leader systems suffer badly because all writes depend on central leader.

But multi-leader systems continue working independently.

Later:

changes synchronize after network recovers

Biggest Problem of Multi-Leader

The PDF strongly emphasizes this:

Write Conflicts

Lecture 02 - 4601

Why Conflicts Happen

Because:

multiple leaders accept writes simultaneously

Two users may modify same data at same time.

Example

Suppose original title:

A

At same time:

Leader 1 → changes A → B

Leader 2 → changes A → C

Both updates succeed locally.

Later replication happens.

Now conflict exists:

B vs C

Lecture 02 - 4601

Why Single-Leader Avoids This

Single leader processes writes sequentially.

So ordering is clear.

But multi-leader has:

concurrent writes

with no global ordering.

Conflict Detection

Conflicts are usually detected:

Asynchronously

Meaning:

1. local write succeeds immediately
2. conflict discovered later during replication

Lecture 02 - 4601

Why Not Detect Conflicts Synchronously?

Because then leaders would need to coordinate before every write.

That destroys the main advantage:

independent local writes

Conflict Resolution Strategies

The PDF discusses several approaches.

1. Conflict Avoidance

Best strategy if possible.

Idea:

same record always written through same leader

Example:

User A always uses Asia leader
User B always uses Europe leader

Then conflicts rarely happen.

Lecture 02 - 4601

Problem

If user moves datacenter,
or datacenter fails,
conflicts can still happen.

2. Last Write Wins (LWW)

Each write gets timestamp.

Newest timestamp wins.

Older value discarded.

Example:

B at 10:01
C at 10:02

Keep C.

Discard B.

Problem

✗ data loss

Potentially dangerous.

3. Merge Values

Combine conflicting values.

Example:

B + C → "B/C"

Useful for some applications.

4. Store Conflicts Explicitly

Database preserves both versions.

Application resolves conflict later.

Sometimes user chooses manually.

Automatic Conflict Resolution

The PDF mentions advanced techniques:

- CRDTs
- Operational Transformation
- Mergeable Data Structures

Lecture 02 - 4601

These are used in:

- Google Docs
- collaborative editors

- distributed synchronization systems
-

Offline Clients Use Case

Another major use case.

Lecture 02 - 4601

Example

Phone calendar app.

Even offline:

user can edit calendar

Each device acts like a leader.

When internet returns:

devices synchronize changes

This is effectively multi-leader replication.

Why This Is Hard

Different devices may edit same data independently.

Replication lag may be:

hours or days

Conflict resolution becomes critical.

Collaborative Editing Use Case

Google Docs-like systems.

Lecture 02 - 4601

Without Locking

Multiple users edit simultaneously.

Each client becomes local leader temporarily.

Changes synchronize continuously.

Again:

conflict resolution required

Replication Topologies

The PDF explains different ways leaders connect.

Lecture 02 - 4601

1. All-to-All Topology

Every leader talks to every other leader.

Example:

```
A ↔ B
A ↔ C
B ↔ C
```

Best fault tolerance.

2. Circular Topology

Example:

```
A → B → C → A
```

Simple,
but one failure may break replication chain.

3. Star Topology

One root leader forwards updates to others.

Ordering Problem

Multi-leader replication introduces causality problems.

Example from PDF:

```
INSERT arrives late
UPDATE arrives early
```

A follower may receive:

```
UPDATE before INSERT
```

which makes no logical sense.

Lecture 02 - 4601

Why Multi-Leader Is Difficult

Because distributed systems now have:

- concurrent writes
- no global ordering
- asynchronous replication

- conflicts
- causality issues

Important Warning from PDF

The PDF says:

multi-leader replication is often considered dangerous territory.

Lecture 02 - 4601

Because many systems implement conflict handling poorly.

Advantages vs Disadvantages

Advantages	Disadvantages
Faster local writes	Write conflicts
Better availability	Complex conflict resolution
Better datacenter independence	Harder consistency guarantees
Better network partition tolerance	Causality problems
Good for offline systems	More operational complexity

Core Difference from Single-Leader

Single-Leader	Multi-Leader
One write authority	Multiple write authorities
No write conflicts	Conflicts possible
Easier consistency	Harder consistency
Simpler	More complex
Lower availability during partitions	Better partition tolerance

Big Conceptual Shift

Single-leader assumes:

one global write order

Multi-leader abandons that assumption.

Now:

multiple histories evolve concurrently

and system must merge them later.

One-Line Summary

Multi-leader replication allows multiple nodes to accept writes independently and asynchronously replicate changes to each other, improving availability and geographic performance but introducing difficult problems like concurrent write conflicts, causality issues, and complex conflict resolution.

explain Handling Write Conflicts part

Handling Write Conflicts

This section explains the **biggest problem in multi-leader replication**:

Two leaders may modify the same data at the same time.

When that happens:

**database does not know
which value is correct**

This is called:

Write Conflict

Lecture 02 - 4601

Why Conflicts Happen

In multi-leader replication:

multiple leaders accept writes independently

Each leader processes local writes immediately.

Replication between leaders happens:

asynchronously

So two users may change the same record before leaders synchronize.

Example from the PDF

Suppose wiki page title is initially:

A

At the same time:

Leader 1:
A → B

Leader 2:
A → C

Both updates succeed locally.

Later leaders exchange updates.

Now conflict exists:

B vs C

Lecture 02 - 4601

Why Single-Leader Avoids This

Single leader processes writes sequentially:

```
write 1
then
write 2
```

So ordering is clear.

But multi-leader allows:

concurrent writes

with no single global ordering.

What Makes This Hard?

At Leader 1:

B happened after A

At Leader 2:

C happened after A

But neither leader knows:

whether B or C came first globally

Because both happened independently.

Synchronous vs Asynchronous Conflict Detection

The PDF compares two approaches.

Lecture 02 - 4601

In Single-Leader Systems

Second writer usually:

- waits

- or transaction aborts

Example:

User 1 editing
User 2 waits

Conflict avoided immediately.

In Multi-Leader Systems

Both writes succeed immediately.

Conflict discovered later during replication.

This is:

Asynchronous Conflict Detection

Why Not Detect Conflicts Immediately?

Because then leaders would need global coordination before every write.

That destroys the purpose of multi-leader replication:

independent local writes

Simplest Strategy: Conflict Avoidance

The PDF says the easiest approach is:

Avoid conflicts completely.

Lecture 02 - 4601

Core Idea

Ensure all writes for a record go through same leader.

Example

Suppose each user has a “home datacenter”:

User A → Asia leader
User B → Europe leader

All writes from User A always go through Asia.

Then concurrent updates become unlikely.

Why This Works

From one user's perspective:

system behaves almost like single-leader

Problem

Conflict avoidance breaks when:

- datacenter fails
- traffic rerouted
- user moves geographically

Then multiple leaders may modify same record again.

Conflict Resolution

When conflicts happen:

database must converge

Meaning:

All replicas must eventually agree on one final state.

Lecture 02 - 4601

Why Convergence Matters

Without convergence:

Leader 1 → B forever
Leader 2 → C forever

Database remains permanently inconsistent.

Not acceptable.

Conflict Resolution Techniques

The PDF explains several methods.

1. Last Write Wins (LWW)

Most common but dangerous.

How It Works

Each write gets unique ID or timestamp.

Example:

B → timestamp 10:01
C → timestamp 10:02

Largest timestamp wins.

Result:

Keep C
Discard B

Lecture 02 - 4601

Advantage

Very simple.

Easy to implement.

Major Problem

✗ Data loss

One valid update disappears completely.

Example

Suppose:

User 1 updates phone number
User 2 updates address

If timestamps conflict:

one update may vanish

Important Insight

The PDF says LWW is:

dangerously prone to data loss

Lecture 02 - 4601

2. Replica Priority

Another approach:

Each replica gets ID.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Example:

Leader A = priority 1
Leader B = priority 2

Higher-priority replica always wins.

Problem

Still causes:

✗ data loss

Lower-priority writes discarded.

3. Merge Values Together

Instead of discarding values:

combine them

Example:

B + C → "B/C"

Advantage

No information lost.

Problem

Merged value may look strange.

Not always meaningful.

4. Store All Conflicts Explicitly

Instead of resolving automatically:

preserve all versions

Application resolves later.

Lecture 02 - 4601

Example

Database stores:

Version 1 = B
Version 2 = C

Later:

- user chooses correct value
 - application merges intelligently
-

Conflict Resolution Timing

The PDF explains two moments when resolution may happen.

A. On Write

Conflict detected immediately during replication.

Database calls custom conflict handler.

Example:

custom Perl script

Handler resolves automatically.

Lecture 02 - 4601

Limitation

Must execute quickly.

Cannot ask user interactively.

B. On Read

Database stores all conflicting versions.

Next read returns all versions.

Application resolves conflict later.

Example:

[B, C]

Application decides final result.

CouchDB works like this.

Lecture 02 - 4601

Important Point About Transactions

Conflict resolution usually works:

per row / per document

NOT for entire transaction.

Lecture 02 - 4601

So even if transaction updates multiple rows atomically:

conflicts handled separately

Automatic Conflict Resolution

The PDF discusses advanced research systems.

Lecture 02 - 4601

1. CRDTs

Conflict-Free Replicated Data Types.

Special data structures that merge automatically.

Examples:

- counters
- sets
- maps
- ordered lists

Example

Two users add items simultaneously:

A adds milk

B adds eggs

CRDT automatically merges:

[milk, eggs]

without conflict.

2. Mergeable Data Structures

Similar to Git merging.

Track history explicitly.

Use multi-way merge algorithms.

3. Operational Transformation

Used in:

- Google Docs

- Etherpad

Designed for collaborative editing.

Allows concurrent edits to same document.

Important Question

What Actually Counts as a Conflict?

The PDF explains this carefully.

Lecture 02 - 4601

Obvious Conflict

Same field updated differently:

A → B
A → C

Easy to detect.

Subtle Conflict Example

Meeting room booking system.

Two users book same room simultaneously.

Even though records differ:

booking overlaps

This is logically a conflict.

Harder to detect automatically.

Why Conflict Detection Is Difficult

Conflicts are often:

application-specific

Database may not understand business logic.

Big Conceptual Shift

Single-leader systems avoid concurrency by enforcing one write order.

Multi-leader systems allow:

multiple concurrent histories

Then later try to merge them safely.

Core Tradeoff

Multi-leader replication improves:

- availability
- latency
- offline operation

But sacrifices:

simple consistency

Visual Summary

```

Leader A: A → B
Leader B: A → C
      ↓
Conflict detected
      ↓
Resolve conflict
      ↓
All replicas converge
  
```

Main Conflict Resolution Strategies

Strategy	Advantage	Problem
Avoid conflicts	Simple	Not always possible
Last Write Wins	Easy	Data loss
Replica priority	Deterministic	Data loss
Merge values	Preserves data	Complex
Store conflicts	Flexible	Application complexity
CRDTs	Automatic merging	Advanced & specialized

Most Important Insight

The hardest part is NOT replication itself.

The hardest part is:

deciding what the “correct” final value should be.

Because when writes are concurrent:

there may be no objectively correct ordering

One-Line Summary

Handling write conflicts in multi-leader replication means detecting concurrent updates to the same data and resolving them—through avoidance, overwriting, merging, or application-level logic—so that all replicas eventually converge to a consistent state.

explain Multi-Leader Replication Topologies from the slide in details

Multi-Leader Replication Topologies

This section explains:

How multiple leaders are connected and how replication messages travel between them.

Lecture 02 - 4601

In multi-leader replication:

many leaders accept writes

So the system must decide:

How should leaders communicate with each other?

That communication structure is called:

Replication Topology

What Is a Topology?

A topology describes:

which leader sends updates to which other leader

It defines the paths along which replication messages travel.

Simple Two-Leader Case

If there are only 2 leaders:

Leader A ↔ Leader B

There is only one obvious design:

- A sends updates to B
- B sends updates to A

Easy.

But With Many Leaders...

Things become more complicated.

Example:

A
B
C
D

Now many connection patterns become possible.

The PDF discusses 3 major topologies:

1. All-to-All
2. Circular
3. Star / Tree

Lecture 02 - 4601

1. All-to-All Topology

Also called:

Fully Connected Topology

Structure

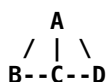
Every leader directly communicates with every other leader.

Example:

A ↔ B
A ↔ C
A ↔ D
B ↔ C
B ↔ D
C ↔ D

Every node has direct connections to all others.

Visual



All nodes interconnected.

How Replication Works

Suppose:

Leader A receives write

A immediately sends update directly to:

- B
- C
- D

No intermediate forwarding needed.

Advantages

Best Fault Tolerance

If one connection fails:

messages can take another path

Example:

A cannot reach C directly

Still possible:

A → B → C

No Single Point of Failure

System remains robust.

Faster Synchronization

Direct communication reduces forwarding delays.

Disadvantages

Many Connections

Number of links grows rapidly.

If there are N leaders:

connections $\approx N^2$

Complexity becomes large.

Ordering Problems

This is the biggest issue discussed in the PDF.

Lecture 02 - 4601

Messages may arrive in different orders.

Example from PDF

Suppose:

Leader 1:
INSERT row

Leader 3:
UPDATE same row

Because network speeds differ:

Leader 2 may receive:

UPDATE before INSERT

Now:

trying to update row that doesn't exist

This creates causality problems.

Why Ordering Breaks

Different network links have different delays.

Example:

A → B = fast
A → C = slow

Messages can “overtake” each other.

Important Concept: Causality

Some operations depend on earlier operations.

Example:

INSERT before UPDATE

UPDATE only makes sense after INSERT.

If ordering breaks:

system becomes inconsistent

Why Timestamps Are Not Enough

The PDF emphasizes:

clocks cannot be trusted perfectly

Different servers have clock skew.

So timestamps alone cannot reliably order events.

Solution Mentioned

The PDF mentions:

Version Vectors

These track causal relationships between writes.

We study them later in leaderless replication.

2. Circular Topology

Also called:

Ring Topology

Structure

Each node sends updates to exactly one other node.

Example:

A → B → C → D → A

Replication forms a loop.

Visual

```

A → B
↑   ↓
D ← C
  
```

How Replication Works

Suppose:

A receives write

Then:

```

A sends to B
B forwards to C
C forwards to D
  
```

Eventually all leaders receive update.

Important Point

Nodes must forward writes they receive from others.

Not just their own writes.

Preventing Infinite Loops

Without protection:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A \rightarrow B \rightarrow \dots$

message would circulate forever.

Solution

Each write carries metadata:

which nodes already processed it

When node sees its own ID:

ignore message

Lecture 02 - 4601

Advantages

Simpler Network Structure

Fewer connections than all-to-all.

Lower Communication Cost

Each node only maintains one outgoing replication stream.

Disadvantages

Poor Fault Tolerance

This is the major weakness.

Suppose:

Node B fails

Then:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

A cannot send updates to C or D

Replication chain breaks.

Lecture 02 - 4601

Important Consequence

One node failure may interrupt replication for entire cluster.

Reconfiguration Problem

Topology can be repaired manually:

A → C directly

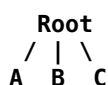
But most systems require manual intervention.

3. Star Topology

Structure

One central root node distributes updates.

Example:



All replication passes through central node.

How It Works

Suppose:

Leader A receives write

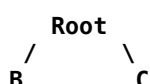
Flow:

A → Root → B and C

Root forwards updates to everyone.

Tree Topology

Star can expand into hierarchical tree:





Messages travel through branches.

Advantages

Simpler Coordination

Central routing easier to manage.

Fewer Connections

Compared to all-to-all.

Disadvantages

Root Node Becomes Critical

If root fails:

replication disrupted

Single point of failure.

Comparison of Topologies

Topology	Structure	Strength	Weakness
All-to-All	Everyone connected	Best fault tolerance	Complex ordering
Circular	Ring	Simple	One failure breaks chain
Star	Central root	Easy management	Root failure dangerous

Core Distributed Systems Problem

The PDF emphasizes that topology design affects:

- fault tolerance
- ordering guarantees
- replication speed
- consistency

Most Important Issue: Message Ordering

In distributed systems:

network delays are unpredictable

So different replicas may observe writes in different orders.

This creates:

Causality Problems

Example

Correct logical order:

```
INSERT user  
UPDATE user
```

But another leader may receive:

```
UPDATE first  
INSERT later
```

This breaks consistency.

Why This Is Hard

There is:

no single global clock

No perfect universal ordering exists naturally.

Important Warning from PDF

The PDF says many multi-leader systems implement conflict detection poorly.

Lecture 02 - 4601

Meaning:

real systems may silently corrupt data

if topology and causality are mishandled.

Big Conceptual Insight

Replication topology is not just network design.

It fundamentally affects:

correctness of distributed replication

Visual Summary

All-to-All

$A \leftrightarrow B \leftrightarrow C \leftrightarrow D$

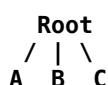
Fast, fault-tolerant, complex.

Circular

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Simple but fragile.

Star



Centralized but risky.

One-Line Summary

Multi-leader replication topologies define how leaders exchange updates, and different designs such as all-to-all, circular, and star topologies trade off simplicity, fault tolerance, communication cost, and the difficulty of maintaining correct ordering and causality across replicas.

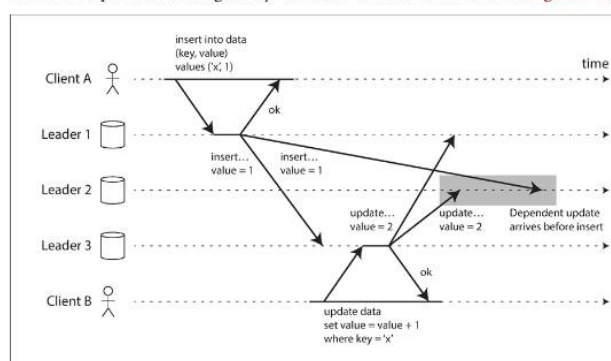


Figure 5-9. With multi-leader replication, writes may arrive in the wrong order at some replicas.

explain this

This figure explains a very important problem in **multi-leader replication**:

Writes can arrive at replicas in the wrong order.

Because leaders replicate asynchronously, network delays may cause one replica to receive an UPDATE before receiving the INSERT that the UPDATE depends on.

That is exactly what this figure demonstrates.

First Understand the Layout

The diagram has:

Client A
Leader 1
Leader 2
Leader 3
Client B

Time moves:

left → right

Each horizontal dotted line is the timeline for one participant.

Step 1 — Client A Inserts Data

At the top:

Client A sends:

```
SQL
INSERT INTO data(key, value)
VALUES ('x', 1);
```

to:

Leader 1

This means:

```
create row:
key = x
value = 1
```

After Insert

Leader 1 now stores:

key	value
x	1

Leader 1 replies:

OK

to Client A.

So insertion succeeded.

Step 2 — Leader 1 Replicates INSERT

Leader 1 now sends replication messages to:

- Leader 2
- Leader 3

The arrows labeled:

`insert... value = 1`

represent replication traffic.

Important Detail

These replication messages travel through the network independently.

Meaning:

they may arrive at different times

There is NO guarantee all replicas receive them simultaneously.

Step 3 — Leader 3 Receives INSERT

Leader 3 receives the insert first.

Now Leader 3 also has:

key	value
x	1

Step 4 — Client B Updates Data

Now Client B sends to Leader 3:

```
SQL
UPDATE data
SET value = value + 1
WHERE key = 'x';
```

Since Leader 3 already has:

`x = 1`

the update succeeds.

Now Leader 3 changes:

x = 2

Leader 3 replies:

OK

to Client B.

Step 5 — Leader 3 Replicates UPDATE

Leader 3 now sends replication messages:

update... value = 2

to other leaders.

Critical Problem Happens Here

Look at the gray shaded area.

At:

Leader 2

the UPDATE arrives BEFORE the INSERT.

This is the central problem.

What Leader 2 Sees

Leader 2 first receives:

UPDATE x → 2

But Leader 2 has never seen:

INSERT x = 1

So from Leader 2's perspective:

row x does not exist yet

Logical Problem

The UPDATE depends on the INSERT.

Correct order should be:

1. **INSERT x=1**
2. **UPDATE x=2**

But Leader 2 observes:

```
1. UPDATE x=2  
2. INSERT x=1
```

This violates causality.

Why Did This Happen?

Because replication is:

asynchronous

Different network paths have different delays.

Example:

```
Leader1 → Leader2 = slow  
Leader3 → Leader2 = fast
```

So UPDATE message “overtook” INSERT message.

This Is Called

Message Reordering

or

Causality Violation

Important Insight

The UPDATE is:

causally dependent

on the INSERT.

Because:

```
you cannot update something  
before it exists
```

Why Single-Leader Systems Don't Have This Problem

In single-leader replication:

```
all writes pass through one ordered log
```

Followers replay writes in same order.

So replicas always see:

```
INSERT before UPDATE
```

But Multi-Leader Has No Single Global Order

Each leader independently generates writes.

Network delays reorder messages.

Therefore:

different replicas may observe different histories

Why This Is Dangerous

If database applies UPDATE before INSERT:

Possible outcomes:

- update fails
- row becomes corrupted
- replicas diverge
- conflicts occur

How Systems Handle This

Different databases use different strategies.

Strategy 1 — Buffer Dependent Updates

Leader 2 may temporarily hold UPDATE:

wait until INSERT arrives

Then apply safely.

Strategy 2 — Version Vectors

Track causal dependencies between writes.

Database knows:

UPDATE depends on INSERT

So ordering enforced logically.

Strategy 3 — Conflict Resolution

If ordering breaks:

resolve inconsistency later

using merge rules.

Key Distributed Systems Lesson

This figure demonstrates:

Network communication is not ordered globally.

Even if operations happen in one logical order:

INSERT → UPDATE

another replica may observe:

UPDATE → INSERT

because distributed systems have:

- unpredictable delays
- asynchronous replication
- no perfect global clock

Most Important Concept in This Figure

The figure is fundamentally about:

Causality

The UPDATE logically depends on the INSERT.

Multi-leader replication may violate causal ordering if replication messages arrive out of order.

Simple Real-World Analogy

Imagine:

1. Someone sends:

"Create folder"

2. Then sends:

"Put file into folder"

If second message arrives first:

folder doesn't exist yet

Same problem.

Final Big Picture

This figure shows why multi-leader replication is difficult:

independent leaders
 +
asynchronous replication
 +
network delays
 =
ordering problems

And ordering problems lead to:

- conflicts
- inconsistency
- causality violations

One-Line Summary

The figure shows that in multi-leader replication, replication messages can arrive at different replicas in different orders, causing causally dependent operations (like UPDATE after INSERT) to be observed incorrectly as UPDATE before INSERT.

explain leaderless replication properly from the slide

Leaderless Replication

So far we studied:

- Single-Leader Replication
- Multi-Leader Replication

Both systems still had the idea of:

some node acts as leader

But leaderless replication removes that completely.

No leader exists.

Any replica can accept reads and writes.

Lecture 02 - 4601

Core Idea

Instead of:

one leader coordinates replication

clients directly communicate with multiple replicas.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Each replica is equal.

Historical Background

The PDF mentions this idea was first used in:

Dynamo (Amazon)

and later influenced:

- Cassandra
- Riak
- Voldemort

Lecture 02 - 4601

Basic Architecture

Suppose we have 3 replicas:

Replica A
Replica B
Replica C

There is:

NO master
NO primary
NO leader

All replicas are peers.

How Writes Work

When client writes data:

client sends write to multiple replicas

Example:

write x=5

sent to:

- A
- B
- C

Each replica stores data independently.

How Reads Work

Client also reads from multiple replicas.

Then compares responses.

This is important because replicas may disagree temporarily.

Why Leaderless Replication Exists

Main goals:

- high availability
- fault tolerance
- partition tolerance

Especially useful when:

some replicas are down

but system should still operate.

Problem with Single-Leader

Suppose leader crashes.

Then:

writes stop until failover

Leaderless systems avoid this.

Any surviving replica can continue serving requests.

Example Write

Suppose:

3 replicas total

Client writes:

x = 100

Client sends write to all replicas.

Results:

Replica	Result
A	success
B	success
C	failed

Maybe Replica C is temporarily offline.

Important Question

Should write still count as successful?

Leaderless systems say:

YES — if enough replicas accepted it.

This introduces the central concept:

Quorums

Quorum Concept

Suppose:

$n = \text{total replicas}$

Example:

$n = 3$

Write Quorum (w)

Minimum replicas that must confirm a write.

Example:

$w = 2$

Meaning:

write succeeds if 2 replicas confirm

Read Quorum (r)

Minimum replicas read during query.

Example:

$r = 2$

Client reads from 2 replicas.

Important Rule

The PDF gives the famous quorum condition:

w + r > n

Lecture 02 - 4601

Why This Matters

Example:

n = 3
w = 2
r = 2

Then:

w + r = 4 > 3

This guarantees:

read and write sets overlap

At least one replica participating in read must contain latest write.

Example

Suppose write stored on:

A and B

Later read checks:

B and C

Because B overlaps:

latest value discovered

Why Quorums Are Powerful

Even if some replicas fail:

system still works

Example

Suppose:

Replica C down

Still:

- writes can go to A+B
- reads can use A+B

No failover needed.

Sloppy Quorums

The PDF explains an important optimization.

Lecture 02 - 4601

Problem

Suppose replicas responsible for data are unavailable.

Example:

A and B crashed

Should system reject writes?

Traditional quorum:

YES

Dynamo-Style Solution

Use:

Sloppy Quorum

Meaning:

store write on temporary substitute nodes

Even if they are not the “correct” replicas.

Example

Original replicas:

A B C

Unavailable:

A B

System temporarily stores write on:

D E

This improves availability.

Hinted Handoff

Later when A/B recover:

temporary nodes forward writes back.

This is called:

Hinted Handoff

Lecture 02 - 4601

Reading Stale Values

Because replication is asynchronous:
replicas may disagree.

Example

Suppose:

A = 5
B = 5
C = 3

C missed latest update.

Now client reads from:

B and C

Which value is correct?

Read Repair

Client notices inconsistency:

B=5
C=3

Client updates stale replica C.

This is:

Read Repair

Lecture 02 - 4601

Anti-Entropy Process

Background synchronization also runs continuously.

Replicas compare data and repair differences automatically.

Often implemented using:

- Merkle Trees
- hash comparison

Problem: Concurrent Writes

Without leader:

two clients may write simultaneously

Example:

Client 1 → x=5
Client 2 → x=8

Different replicas may receive different writes first.

Now replicas disagree.

Last Write Wins (LWW)

One common approach:

latest timestamp wins

But this causes:

✗ data loss

Version Vectors / Vector Clocks

The PDF introduces these for conflict detection.

Why Needed

System must determine:

Which writes happened before others?

or

Were writes concurrent?

Example

Suppose:

Version 1 → x=5
Version 2 → x=8

If neither descends from other:

they are concurrent versions

Conflict exists.

Siblings

Leaderless databases may preserve multiple versions simultaneously.

Example:

`x = [5, 8]`

These versions are called:

Siblings

Application later merges them.

Eventual Consistency

Leaderless systems usually provide:

Eventual Consistency

Meaning:

replicas may temporarily disagree

but eventually converge.

Important Tradeoff

Leaderless replication prioritizes:

availability

over:

strong consistency

Especially during:

- node failures
- network partitions

CAP Perspective

Leaderless systems are designed to continue operating during partitions.

Meaning they strongly favor:

AP systems

in CAP theorem terms.

Comparison with Leader-Based Systems

Feature	Leader-Based	Leaderless
Leader exists	Yes	No
Writes go through leader	Yes	No
Failover needed	Yes	No
Conflict handling	Centralized	Distributed
Availability	Lower during failures	Higher
Consistency	Easier	Harder

Big Conceptual Shift

Leader-based systems assume:

single ordered history

Leaderless systems allow:

many replicas evolving independently

Then try to reconcile later.

Visual Example

Client

↓

Writes to A B C

A=5

B=5

C=3

Read from B+C

Detect inconsistency

Repair C

Most Important Ideas

Leaderless replication revolves around:

1. Quorums
2. Eventual consistency
3. Conflict resolution
4. Replica repair
5. Availability during failures

One-Line Summary

Leaderless replication removes the leader entirely and allows any replica to accept reads and writes, using quorum reads/writes, replica repair, and conflict resolution techniques to achieve high availability and eventual consistency despite failures and concurrent updates.